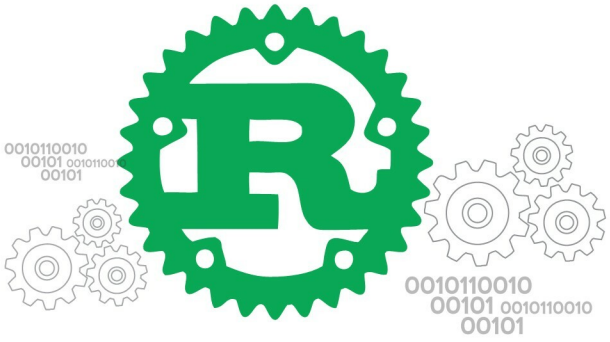
Begin Programming *with* Rust

Rust has had its beginnings in the labs of Mozilla Research. This open source programming language is extremely fast and has become very popular in recent years.



As defined by the developers themselves, Rust is a systems programming language that runs blazingly fast, prevents segfaults, and guarantees thread safety. Rust is sponsored by Mozilla Research which describes it as a safe, concurrent and practical language.

Why learn Rust?

The goal of those who developed Rust was for it to be a good language for creating highly concurrent and safe systems. Rust is also designed to provide speed and safety, at the same time. Due to these features, Rust has held the title of 'Most Loved Programming Language' in the Stack Overflow Developer survey for three straight years, starting from 2016 to 2018.

What is so unique about Rust?

Rust is intended to be a language for highly concurrent and safe systems and "programming in the large," that is, creating and maintaining boundaries that preserve large-system integrity. This has led to a feature set with an emphasis on safety, control of memory layout and concurrency. The performance of idiomatic Rust is comparable to the performance of idiomatic C++. Some other unique features of Rust are:

- Minimal runtime
- Efficient C bindings
- Efficient thread handling without race conditions between the shared data
- Guaranteed memory safety
- Type inference

Let's begin with 'Hello World!'

In the age-old tradition of programming, let's first greet the world with a 'Hello', i.e., a 'Hello World' program, as given below:

```
fn main() {
    println!("Hello, world!");
}
```

There is quite a lot going on in these two lines of code. Let's begin with *fn*, which is how a function is defined in Rust. *main* is the entry point of every Rust program. *println!* prints text to the console and its *!* indicates that it's a macro instead of a function. A macro in Rust is a way to encapsulate patterns that simply can't be done using a function abstraction.